

React JS Hands-On Lab

Component Life Cycle Methods – Blog App

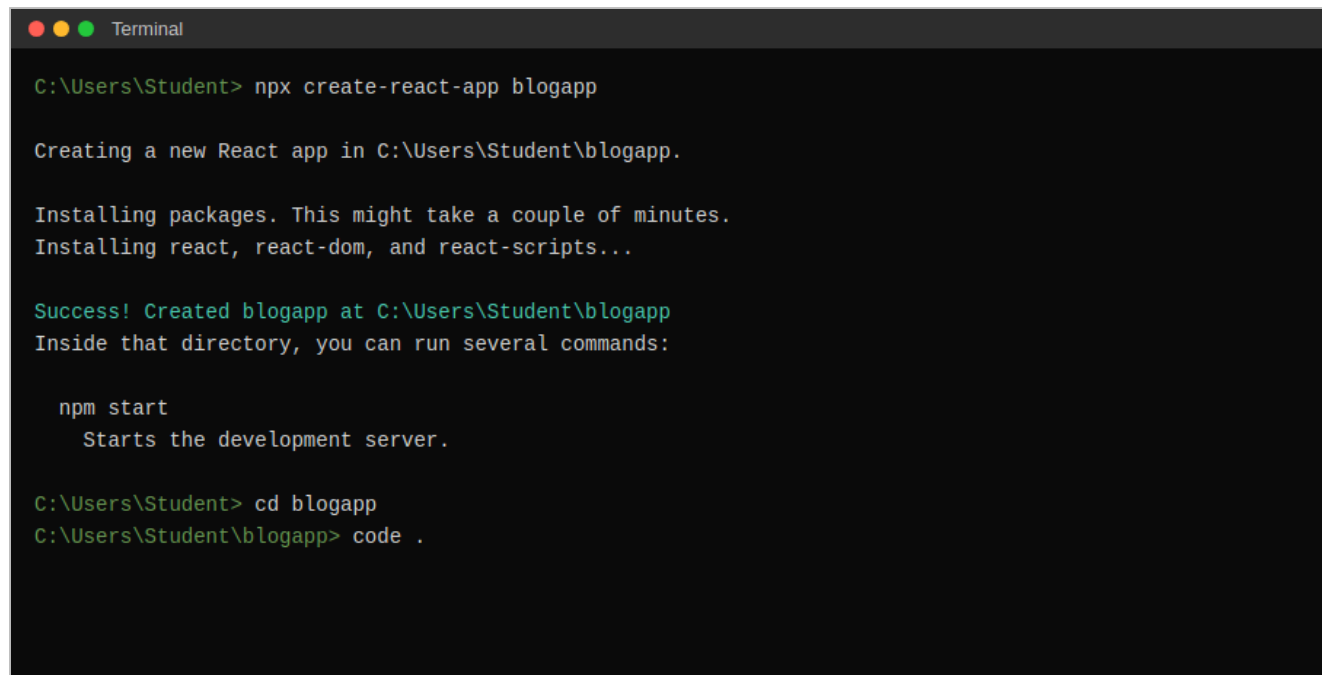
Objectives

- Explain the need and benefits of component life cycle
- Identify various life cycle hook methods
- List the sequence of steps in rendering a component

In this lab I implemented the `componentDidMount()` hook and the `componentDidCatch()` life cycle hook.

Step 1: Create a new React app named "blogapp"

Command used: `npx create-react-app blogapp`



```
Terminal

C:\Users\Student> npx create-react-app blogapp

Creating a new React app in C:\Users\Student\blogapp.

Installing packages. This might take a couple of minutes.
Installing react, react-dom, and react-scripts...

Success! Created blogapp at C:\Users\Student\blogapp
Inside that directory, you can run several commands:

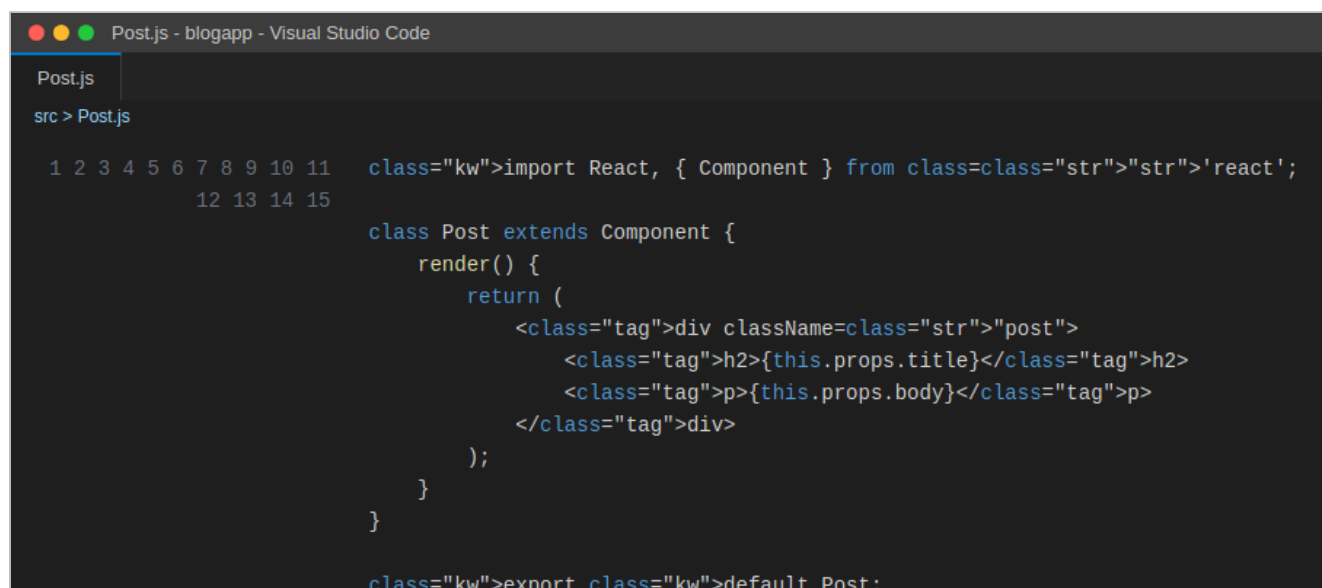
  npm start
    Starts the development server.

C:\Users\Student> cd blogapp
C:\Users\Student\blogapp> code .
```

Step 2: Open the project in VS Code

Command used: `code .` (shown in the terminal screenshot above)

Step 3: Create Post.js in the src folder



```
Post.js
src > Post.js

1 2 3 4 5 6 7 8 9 10 11  class="kw">import React, { Component } from class="str">"str">'react';
12 13 14 15
    class Post extends Component {
      render() {
        return (
          <class="tag">div className=class="str">"post">
            <class="tag">h2>{this.props.title}</class="tag">h2>
            <class="tag">p>{this.props.body}</class="tag">p>
          </class="tag">div>
        );
      }
    }

    class="kw">export class="kw">default Post;
```

Step 4: Create Posts.js and initialize state in the constructor

```
Posts.js
src > Posts.js

1 2 3 4 5 6 7 8 9 10 11  class="kw">import React, { Component } from class="str">"str">'react';
12 13  class="kw">import Post from class="str">"str">'./Post';

    class Posts extends Component {
      constructor(props) {
        super(props);
        this.state = {
          posts: []
        };
      }
    }

    class="kw">export class="kw">default Posts;
```

Step 5: Create loadPosts() method using the Fetch API

URL used: <https://jsonplaceholder.typicode.com/posts>

```
Posts.js
src > Posts.js

1 2 3      loadPosts() {
4 5 6        fetch(class="str">"str">'https:cmnt">"/jsonplaceholder.typicode.com/posts')
7 8 9        .then(response => response.json())
10         .then(data => {
            this.setState({ posts: data });
          })
        .catch(error => {
          console.log(error);
        });
      }
    }
```

Step 6: Implement componentDidMount() to call loadPosts()

```
Posts.js
src > Posts.js

1 2 3      componentDidMount() {
          this.loadPosts();
        }
    }
```

Step 7: Implement render() to display title and body of each post

```
Posts.js - blogapp - Visual Studio Code
Posts.js
src > Posts.js

1 2 3 4 5      render() {
6 7 8 9          return (
                    <class="tag">div>
                      {this.state.posts.map(post => (
                        <class="tag">Post key={post.id} title={post.title} body={post.body} />
                      )))
                    </class="tag">div>
                );
            }
        }
```

Step 8: Implement componentDidCatch() to show errors as alerts

```
Posts.js - blogapp - Visual Studio Code
Posts.js
src > Posts.js

1 2 3      componentDidCatch(error, info) {
            alert(class="str">"Something went wrong: " + error);
        }
```

Step 9: Add Posts component to App.js

```
App.js - blogapp - Visual Studio Code
App.js
src > App.js

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15  class="kw">import React from class="class="str">"str">'react';
                                   class="kw">import Posts from class="class="str">"str">'./Posts';
                                   class="kw">import class="class="str">"str">'./App.css';

                                   function App() {
                                       return (
                                           <class="tag">div className=class="str">"App">
                                               <class="tag">h1>Blog App</class="tag">h1>
                                               <class="tag">Posts />
                                           </class="tag">div>
                                       );
                                   }
```

Step 10: Build and run using npm start

```
Terminal
C:\Users\Student\blogapp> npm start

Compiling...

Compiled successfully!

You can now view blogapp in the browser.

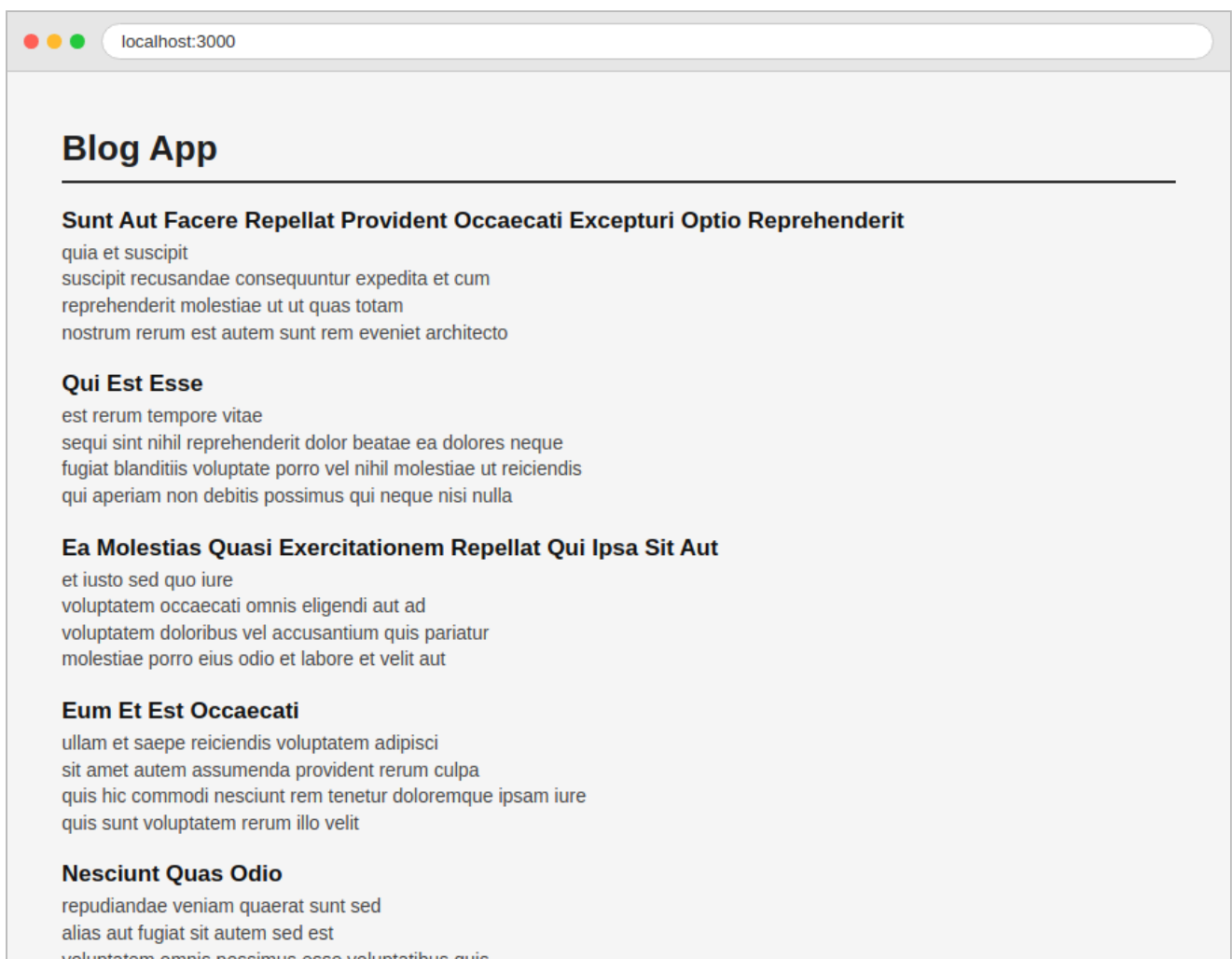
Local:      http://localhost:3000
On Your Network:  http://192.168.1.7:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
```

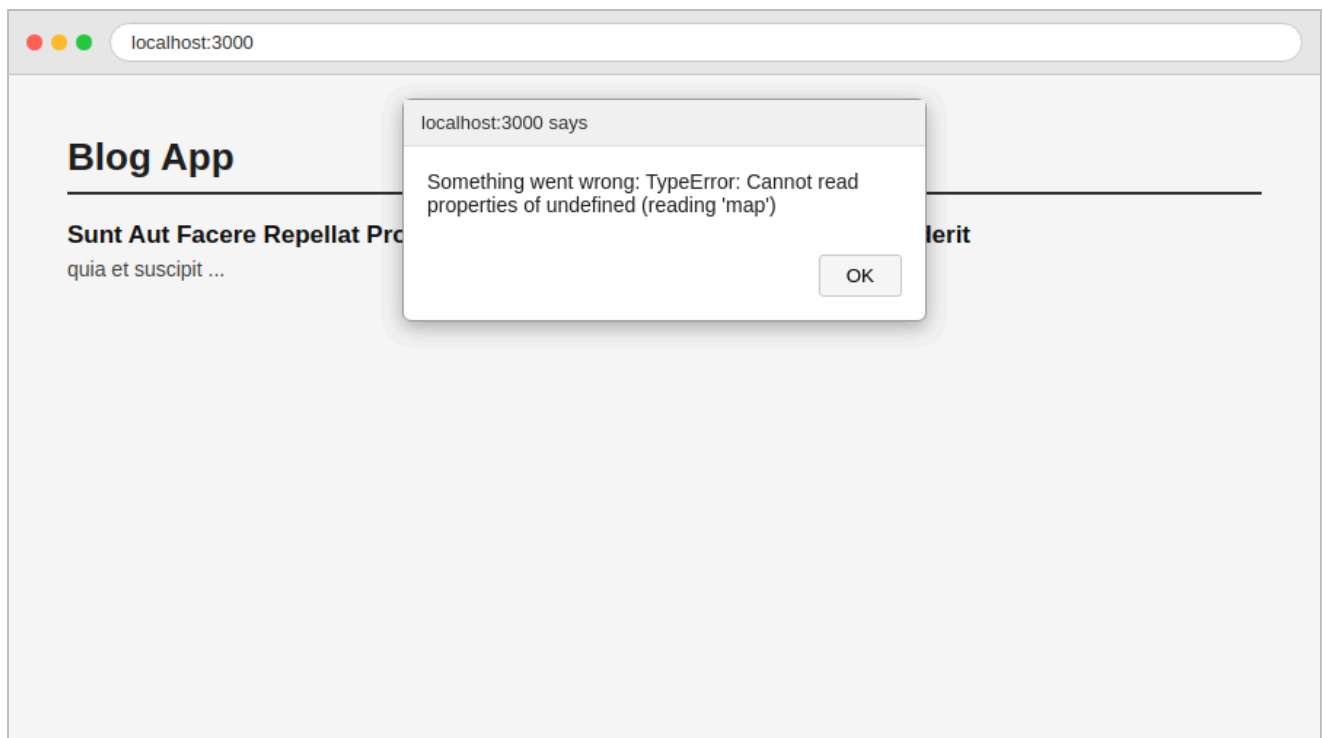
Output: Blog App in the browser

The posts fetched from the API are displayed with title as heading and body as paragraph.



Output: componentDidCatch() showing an error as an alert

When an error occurs while rendering, componentDidCatch() catches it and shows it as an alert message.



Result

The React component life cycle hooks `componentDidMount()` and `componentDidCatch()` were implemented successfully and the Blog App was run and verified.